

Preregistered Paired Evaluation of a Deterministic Verifier Pipeline for LLM-Generated Network Configuration Changes — Non-informative Result under Shared-Generation Semantics

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered paired experiment (cand-2026-01-prereg-v1) that tested whether a deterministic verifier pipeline (generate → deterministic_netops_validator_v5 → optional repair) reduces the rate of verifier-detected unsafe intent→configuration proposals produced by a hosted LLM on synthetic NetOps tasks. Design: N=200 preregistered unique intent→configuration tasks in a paired design comparing (a) baseline LLM-only and (b) guarded pipeline (gpt-5-mini + deterministic_netops_validator_v5). Execution used shared_initial_candidate generation semantics (one LLM call per task) producing model_calls_used = 200 (preregistered independent per-arm generation would have used up to 400). Observed (adversarial cluster): baseline SAFE = 200/200; guarded SAFE = 200/200 (paired contingency n11 = 200, n10 = 0, n01 = 0). Estimated paired SAFE-rate difference = 0.0; exact McNemar two-sided $p = 1.0$; paired bootstrap 95% CI = [0.0, 0.0] (resamples = 10000, seed = 1729). Because identical per-pair generations were produced and the observed baseline unsafe rate was 0%, the preregistered causal hypothesis (that the deterministic verifier reduces unsafe proposals) was not testable in this execution. We publish the complete preregistered run and artifacts, provide an artifact-grounded diagnosis of testability failure, and give concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial injection calibration, and exploratory ROC reserves) for informative repeats. All execution and analysis artifacts cited here are archived in the supplied bundle.

I. INTRODUCTION

Automating human-intent → device-configuration translation with large language models (LLMs) is an active NetOps research thrust because it can reduce human effort and speed maintenance tasks. Yet incorrectly specified or misordered changes can cause outages, policy violations, or security exposures. A commonly proposed mitigation is tool-augmentation: couple an LLM generator to deterministic validators, sandboxed simulators, or constrained executors in a generate-validate-repair pipeline so that proposed changes are checked before execution.

We preregistered and executed a confirmatory paired experiment (cand-2026-01-prereg-v1) to quantify whether deterministic verifier augmentation causally reduces the frequency of verifier-detected unsafe proposals from a hosted LLM under both benign and adversarial prompt clusters. The preregistered estimand was the paired SAFE-rate difference (guarded minus

baseline), tested by an exact McNemar two-sided test with a paired bootstrap CI (10000 resamples, seed = 1729). The design sampled N=200 tasks using a deterministic generator and a paired comparison across two pipelines. Two generation semantics were permitted in the preregistration: independent per-arm generation (one LLM call per arm per task) or shared_initial_candidate (one shared LLM call per task scored by both arms). The executed run used shared_initial_candidate to conserve model-call budget.

Key outcome: under the executed shared semantics the sampled LLM outputs were scored SAFE by the deterministic validator in every confirmatory episode (baseline SAFE = 200/200; guarded SAFE = 200/200). This concordant ceiling outcome (n11 = 200; n10 = n01 = n00 = 0) yields an estimate of zero paired difference and an exact McNemar $p = 1.0$. Because the observed baseline unsafe rate was 0% and the same candidate was used in both arms, the preregistered causal hypothesis could not be meaningfully evaluated. The manuscript therefore focuses on three contributions grounded in archived artifacts: (1) a complete, deterministic preregistered run published as reproducible artifacts; (2) a precise, artifact-grounded diagnosis of why this execution was non-informative for the confirmatory estimand; and (3) concrete, artifact-supported design recommendations that would enable informative repeats under the same preregistration framework.

II. RELATED WORK

Recent surveys and prototypes document strong interest in applying LLMs to NetOps tasks—intent translation, configuration synthesis, diagnosis, and limited remediation—but emphasize that demonstrated gains are largely in prototype or curated-case settings rather than in production deployments [openalex:W7161354925; openalex:W7170714602; openalex:W4411109869]. Several lines of prior work are directly relevant to our design choices and to the generate-validate-repair architecture we evaluated. First, multiple studies and proposals show that augmenting LLMs with deterministic tools (verifiers, simulators, or formal checkers) reduces certain classes of constraint violations on curated tasks: generate-validate-refine or divide-verify-refine pipelines measurably improve adherence to structured in-

structions when the verifier faithfully captures the task constraints [openalex:W4412888330; openalex:W7161354925]. These works, however, also report that improvement magnitude varies with verifier fidelity and the nature of the constraints being checked.

Second, research on guardrails, tool schemas, and MCP-style governance highlights practical trade-offs encountered by tool-augmented agents: stricter schema enforcement reduces some unsafe outputs but can increase false refusals or operator workload unless tuned carefully [TechRxiv:177006109; TechRxiv:177004277]. Red-teaming and guardrail-distillation literature from adjacent domains shows that adversarial prompts can often be engineered to bypass naive guardrails, motivating replayable, sandboxed evaluations and explicit ROC-style diagnostics for verifiers [openalex:W7172000952; TechRxiv:177006109].

Third, evaluation critiques argue that static benchmarks are insufficient for agentic NetOps: meaningful assessment requires workflow-centred, deterministic replay (sandbox) environments, archived execution traces, and paired causal designs that can separate generator variability from validator effects [openalex:W7161354925; doi:10.2139/ssrn.7268438]. These methodological prescriptions informed our preregistration (paired design, deterministic task generator, and archived validator traces) because they make it possible to ascribe observed outcome differences to pipeline interventions rather than to uncontrolled environmental variability.

Finally, prototype and architecture surveys stress operational integration concerns—human-in-the-loop gates, auditability, multi-tool orchestration, and simulator fidelity—all of which constrain what a verifier can safely assert in practice and how empirical evaluation results should be interpreted for operational readiness [TechRxiv:177004277; openalex:W4406549441]. In sum, prior work supports the conceptual value of verifier augmentation but also warns that empirical gains are task- and verifier-dependent and that evaluation semantics (for example, shared vs independent generation and verifier completeness) strongly affect the interpretability of paired causal contrasts. Our preregistered design attempted to operationalize these recommendations; the archived run and the diagnosis below show how a specific execution semantics (shared_initial_candidate) interacted with task difficulty and verifier outputs to produce a non-informative confirmatory outcome.

III. METHODOLOGY

Preregistration and estimand. The experiment was preregistered as cand-2026-01-prereg-v1. Primary estimand: paired_success_rate_difference_guarded_minus_baseline (guarded - baseline) on binary SAFE indicators obtained from a deterministic sandbox validator. Confirmatory test: exact two-sided McNemar implemented by paired_binary_analysis_v1; confidence intervals: paired bootstrap percentile (10000 resamples, seed = 1729). The preregistration specified a paired binary design with N=200 tasks, planned paired episodes = 400, and a power plan

that targeted detecting an approximate 7 percentage-point reduction in unsafe proposals with 80% power assuming a baseline unsafe rate $\approx 10\text{--}15\%$.

Task generator, scope, and seeds. A deterministic generator (deterministic_netops_task_generator_v5) produced 200 synthetic intent→configuration tasks constrained to single-AS multi-device topologies and limited configuration fields (admin, mtu, vlan). Generator seeds were deterministic (per-task generation_seed 1..200; the RNG seed for the generator family was preregistered) and each task manifest included initial_state, expected_final_state, explicit per-task workflow policies (required_sequence), and difficulty metadata. Tasks were clustered by benign vs adversarial prompt transforms using a preregistered obfuscation template set to increase adversarial difficulty; the adversarial cluster was included in confirmatory items per the preregistration.

Pipelines compared. Baseline: accept the LLM candidate (no runtime deterministic enforcement) and score post hoc with deterministic_netops_validator_v5. Guarded: validate the same LLM candidate in a deterministic sandbox using deterministic_netops_validator_v5; the validator emitted structured traces (validation_before/after, violation lists) and optionally could invoke one automatic repair call per the preregistration. For confirmatory episodes no repairs were applied (repair fields are null in scoring JSONs) so the guarded result reflects pure validation rather than repair-induced change.

Generation semantics and model budgeting. The preregistered plan allowed two generation semantics: independent per-arm generation (two model calls per task) or shared_initial_candidate (one model call per task whose candidate is scored by both arms). The executed run used shared_initial_candidate to conserve hosted model budget. Planned maximum_model_calls for independent generation = 400; executed model_calls_used = 200. This single shared candidate per task meant that, absent repair, the two arms would always score the same candidate and could not produce discordant paired outcomes arising from arm-level stochasticity.

Scoring and analysis pipeline. Deterministic_netops_validator_v5 applied full_intent_constraint_validation to each candidate and emitted a deterministic valid flag and detailed violation lists. Confirmatory binary outcomes were derived from validator.valid (1 = SAFE, 0 = UNSAFE). Analysis used paired_binary_analysis_v1 to compute the 2×2 contingency counts (n11, n10, n01, n00), exact McNemar two-sided p-value, and paired bootstrap CI (10000 resamples, seed = 1729). Missingness and retry rules from the preregistration were applied: one automatic retry per failed model call; contaminated or nondeterministic tasks are excluded and replaced. Deterministic reconciliation checks and provider audit were recorded as analysis artifacts and used to certify that no technical failures, exclusions, or nondeterminism occurred in the confirmatory run.

IV. RESULTS

Execution accounting and provider audit. The execution manifest records `completed_episode_count` = 400 (200 paired episodes), `failed_episode_count` = 0, `model_calls_used` = 200, and `execution_mode` = `scientific_confirmatory`. The `model_configuration` archived the declared model identifier (`gpt-5-mini`) and the executed generation semantics (`independent_condition_generation` = `false`). `Deterministic_reconciliation` documented `hosted_model_call_event_count` = 200 and that all deterministic consistency checks passed; there were zero technical failures and zero exclusions in the confirmatory set.

Primary confirmatory outcomes. Analysis `condition_summary` reports baseline SAFE successes = 200/200 and guarded SAFE successes = 200/200. The paired contingency counts are: `n11` = 200 (SAFE both arms), `n10` = 0, `n01` = 0, `n00` = 0. The paired SAFE-rate difference (`guarded` - `baseline`) = 0.0. Exact McNemar two-sided p = 1.0. Paired bootstrap percentile 95% CI = [0.0, 0.0] (`resamples` = 10000, `seed` = 1729). These values are taken directly from the analysis executor outputs and archived analysis artifacts.

Representative diagnostics and failure accounting. Representative scoring JSONs (e.g., `execution/scoring/task-000001-baseline.json` and `task-000001-guarded.json`) show identical `shared_initial_candidate` content and validator traces with `violation_count` = 0 and `valid` = `true` both before and after validation. Representative `shared_initial` response files (`execution/responses/task-000001-shared-initial.txt`, `task-000002-shared-initial.txt`, `task-000003-shared-initial.txt`) contain canonical generated configurations used by both arms. The `repair_provider_trace` fields are null for confirmatory episodes, confirming that no repair changed any candidate and therefore no repair-driven discordance occurred. Analysis/`contamination_summary` shows no flagged episodes; analysis/`missingness_summary` documents `complete_pairs` = 200 with zero failures.

Exploratory reserves unused. The preregistration reserved up to 50 exploratory tasks to estimate verifier ROC and refusal trade-offs; that reserve was not consumed prior to confirmatory execution. Therefore no empirical verifier ROC or refusal-rate estimates were computed in the archived confirmatory artifacts.

V. DISCUSSION

Artifact-grounded diagnosis of the testability (ceiling) failure. Three concrete archived facts explain why the confirmatory hypothesis was untestable in this execution: (1) the run executed `shared_initial_candidate` semantics (one LLM call per task shared across arms) to conserve budget; (2) the sampled `gpt-5-mini` outputs were labeled SAFE by `deterministic_netops_validator_v5` for every confirmatory task (baseline SAFE = 200/200); and (3) no repair calls were applied in confirmatory episodes, so the guarded arm could not introduce arm-only divergences. Together these facts yield a degenerate paired contingency with no discordant pairs, which makes McNemar inference vacuous (p = 1.0; CI degenerate at 0.0).

Interpretation relative to the preregistered power plan. The preregistration’s power calculation assumed a nonzero baseline unsafe rate ($\approx 10\text{--}15\%$) and discordant proportions that would render a paired reduction (~ 7 percentage points) detectable with $N \approx 174$. The executed shared semantics halved model calls (200 vs planned 400 for independent per-arm draws) and removed arm stochasticity; hence the empirical baseline unsafe rate observed here (0%) contradicts preregistered power assumptions and collapses inferential sensitivity. This is an execution semantic and budgeting failure, not evidence about verifier efficacy.

Concrete, artifact-supported recommendations for informative repeats. All recommendations below follow from archived planning and execution artifacts: - Use independent per-arm generation to enable arm-level stochastic contrasts when the estimand is the effect of adding a deterministic validator to generation. For $N=200$ this requires ≈ 200 extra model calls (executed `model_calls_used` = 200 vs preregistered `maximum_model_calls` = 400). If repairs are not applied, independent draws are necessary to create discordant pairs. - Reserve and use the preregistered exploratory budget (up to 50 tasks) to tune adversarial prompt transforms until a nonzero baseline unsafe rate consistent with power assumptions is observed. Archive ROC/reserve outputs prior to confirmatory execution. - If model-call budgets are constrained, prioritize independent per-arm draws over additional model varieties because paired causal contrasts require per-arm variability more than additional cross-model comparisons within the same budget. - When repairs are allowed in the guarded arm, instrument and archive repair traces and report how often repairs convert UNSAFE $\rightarrow$ SAFE and whether repairs introduce new violations; include `repair_provider_trace` artifacts in analysis. In our confirmatory run repair traces were null and therefore not a source of discordance.

Threats to validity and generalizability. Internal validity: results depend critically on generation semantics and model budgeting; the executed shared semantics created the ceiling failure. Construct validity: `deterministic_netops_validator_v5` encodes a formalized `full_intent_constraint_validation` for our synthetic task family, but its completeness relative to all real-world NetOps failure modes is not established. External validity: tasks are synthetic single-AS topologies limited to admin/mtu/vlan changes and the evaluation used a single declared model (`gpt-5-mini`); results do not generalize to multi-vendor production networks or other LLMs. Conclusion validity: the observed degenerate contingency prevents inferential claims about verifier efficacy in this execution; the statistical machinery produced p = 1.0 and a degenerate CI because there were no discordant observations.

VI. LIMITATIONS

- Synthetic tasks and scope: tasks were deterministic synthetic topologies produced by `deterministic_netops_task_generator_v5` and limited to single-AS, multi-device setups; results may not generalize to production, multi-AS, or vendor-specific features.

- Generation semantics: the executed run used `shared_initial_candidate` (`independent_condition_generation = false`), producing identical per-pair generations and creating a ceiling effect when shared candidates were valid.
- Adversarial strength: the preregistered adversarial templates used in this run did not elicit unsafe outputs from gpt-5-mini on the sampled tasks; stronger adversarial cases or explicit injected unsafe examples are required to measure verifier effect.
- Single-model scope: only the declared model (gpt-5-mini) was evaluated; no multi-model generality is claimed.
- Verifier completeness: `deterministic_netops_validator_v5` ran deterministically in synthetic sandboxed states; external validation of validator completeness against all real-world NetOps failure modes is not provided.
- Operational external validity: no production deployment, operator-in-the-loop workload measurement, nor multi-vendor field trials were performed; operator-cost trade-offs remain unquantified at scale.

VII. CONCLUSION

We executed a fully archived preregistered paired experiment comparing gpt-5-mini baseline generation and a deterministic verifier-augmented pipeline on 200 synthetic NetOps tasks. Under the executed `shared_initial_candidate` semantics and for the declared model, both arms produced zero verifier-detected unsafe proposals (paired difference = 0.0; exact McNemar $p = 1.0$; 95% CI = [0.0, 0.0]). Because identical per-pair generations were used and because the observed baseline unsafe rate was 0%, the preregistered causal hypothesis was not testable in this run. The scientific contribution is therefore methodological and reproducibility-centred: (1) a complete deterministic preregistered run with archived artifacts that permit exact replay; (2) an artifact-grounded diagnosis of why this execution was non-informative for verifier efficacy; and (3) concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial calibration, and exploratory ROC estimation) for informative repeats. The archived execution and analysis artifacts cited throughout (`execution/*` and `analysis/*`) permit deterministic replication of the diagnosis and the run outputs and should be consulted prior to attempting repeats or production extrapolation.

DISCLOSURE STATEMENT

Disclosure Statement (CNSM Agentic AI Researcher track): Declared LLM: gpt-5-mini (execution recorded in `execution/model_configuration.json`). Orchestration/framework: `hosted_netops_gvr_v1` adapter and deterministic experiment harness (`execution/execution_manifest.json`). Exact initial master prompt: archived at `provenance/master_prompt.txt` with immutable SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39015ba. Topic constraints and autonomy boundary: the human Research Director selected the broad topic family (Generative AI and LLMs for NetOps) prior to the autonomy lock;

after the lock the autonomous researcher performed literature review, preregistration (`cand-2026-01-prereg-v1`), experiment design, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revisions. Preregistration and execution: the preregistration was frozen before execution; key preregistered elements (estimand, paired design $N=200$, task generator, allowed generation semantics, scoring rules, and stopping rules) are recorded in `cand-2026-01-prereg-v1` and archived in the manuscript bundle. Generation semantics and model-call accounting (executed): `independent_condition_generation = false` (`shared_initial_candidate` semantics) producing a single initial LLM candidate per task shared across arms; `model_calls_used = 200` while the preregistered maximum `model_calls` allocated for independent per-arm generation = 400 (`execution/execution_manifest.json` and `execution/model_configuration.json`). Validator and scoring: `deterministic_netops_validator_v5` performed deterministic sandboxed validation and encoded outcomes as binary labels (1 = SAFE, 0 = UNSAFE) under `scoring_rule = full_intent_constraint_validation`; archived scoring JSONs (`execution/scoring/*json`) contain validator traces showing `validation_before/after` and violation lists. Analysis executor: `paired_binary_analysis_v1` computed the paired contingency, exact McNemar two-sided test, and paired bootstrap CI (resamples = 10000, seed = 1729); analysis artifacts include `analysis/paired_contingency_table.csv` and `analysis/condition_summary.csv`. Deviations and restarts: no post-preregistration design changes were executed; the observed model call count reflects the executed `shared_initial_candidate` semantics. Reproducibility pointers (concise): `execution/raw_results.jsonl`, `execution/task_manifest.jsonl`, `execution/model_configuration.json`, `execution/execution_manifest.json`, `analysis/paired_contingency_table.csv`, `analysis/condition_summary.csv`, and `analysis/paired_difference_figure.svg` are archived in the supplied bundle and permit deterministic replays. Human editing: no manual human edits to technical content, analyses, or manuscript text occurred after the autonomy lock. Pre-lock development: infrastructure rehearsals occurred prior to the lock but were excluded from final empirical evidence unless reproduced under the post-lock execution.

REFERENCES

- [1] <https://arxiv.org/abs/2605.12729>
- [2] <https://doi.org/10.2139/ssrn.7132138>
- [3] <https://doi.org/10.36227/techrxiv.177004277.71795055/v1>
- [4] <https://doi.org/10.36227/techrxiv.177006109.97957916/v1>
- [5] <https://doi.org/10.1109/ojcoms.2026.3680457>
- [6] Xianren Zhang, Xianfeng Tang, Hui Liu, Zongyu Wu, Qi He, Dongwon Lee, Suhang Wang, "Divide-Verify-Refine: Can LLMs Self-align with Complex Instructions?", 2025, 10.18653/v1/2025.findings-acl.709
- [7] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026